INTELLIGENT DOCUMENT VERTIFICATION SYSTEM

A MULTIDISCIPLINARY PROJECT REPORT

Submitted by

Name & Roll No.	*in partial fulfilment of the requirements for the award of the degree of* BACHELOR OF ENGINEERING
LOKGESH B 24ITR067	INFORMATION TECNOLOGY
RANJITH K 24ITR113	INFORMATION TECNOLOGY
KALAIVANI S 24EER053	ELECTRICAL & ELECTRONICS ENGINEERING
SUGANYA S 24EER118	ELECTRICAL & ELECTRONICS ENGINEERING

DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING &
DEPARTMENT OF INFORMATION TECHNOLOGY

KONGU ENGINEERING COLLEGE
TRANSFORM YOURSELF

(Autonomous)
PERUNDURAI ERODE – 638 060
OCTOBER 2026

DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING &
DEPARTMENT OF INFORMATION TECHNOLOGY

KONGU ENGINEERING COLLEGE
(Autonomous)
PERUNDURAI ERODE – 638060
OCTOBER 2026

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "INTELLIGENT DOCUMENT VERIFICATION SYSTEM" is the Bonafide record of project work done by the following students:

Name & Roll No.	*in partial fulfilment of the requirements for the award of the degree of* BACHELOR OF ENGINEERING
LOKGESH B 24ITR067	INFORMATION TECNOLOGY
RANJITH K 24ITR113	INFORMATION TECNOLOGY
KALAIVANI S 24EER053	ELECTRICAL & ELECTRONICS ENGINEERING
SUGANYA S 24EER118	ELECTRICAL & ELECTRONICS ENGINEERING

of Anna university Chennai during the year 2026-2027.

Co-Mentor	Mentor	HoD / Dept of Co-Mentor (Sign. & seal)	HoD/ Dept.of Mentor (Sign. & seal)

Date:

Submitted for the end semester viva voce examination held on _____

INTERNAL EXAMINER EXTERNAL EXAMINER

DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING &
DEPARTMENT OF INFORMATION TECHNOLOGY

KONGU ENGINEERING COLLEGE
(Autonomous)
PERUNDURAI ERODE – 638060
OCTOBER 2026

DECLARATION

We affirm that the Project Report titled "INTELLIGENT DOCUMENT VERIFICATION SYSTEM" being submitted in partial fulfilment of the requirements for the award of Bachelor of Engineering is the original work carried out by us. It has not formed the part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Date:

LOKGESH K
(24ITR067)

RANJITH K
(24EER117)

KALAIVANI S
(24EER053)

SUGANYA S
(24EER118)

I certify that the declaration made by the above candidates is true to the best of my knowledge.

Date:

Name, Dept. and Signature of
the Co-Mentor with seal

Name and Signature of
the Mentor with seal

ABSTRACT

Documents such as mark sheets, degree certificates, identity cards, bonafide letters, and experience letters are required at almost every stage of admission, recruitment, and official verification. Checking these documents manually is slow, depends on the experience of the person verifying them, and is open to human error. At the same time, easy access to image-editing tools has made it simple to alter names, marks, dates, photographs, and seals in scanned documents. A verification process that is fast, consistent, and traceable is therefore needed.

Documents such as identity cards, educational certificates, bank statements, and government-issued identity proofs are required at every stage of admission, recruitment, financial transactions, and official onboarding. Verifying these documents manually is slow, subjective, and prone to human error. Fraudulent documents are increasingly common owing to the easy availability of image-editing tools that can alter names, numbers, photographs, and dates in scanned files.

The project entitled Intelligent Document Verification System (VeriDoc AI) presents a full-stack, AI-assisted web application that automates multi-stage forensic verification of uploaded documents. The system accepts PDF, JPEG, and PNG files and passes them through an eight-stage deterministic pipeline: (1) file integrity validation, (2) multi-engine OCR text extraction using Windows Media OCR with Pillow-enhanced preprocessing and Tesseract fallback, (3) document-type classification with mismatch detection, (4) regex-based field extraction, (5) document-specific rule evaluation including Aadhaar Verhoeff checksum and bank statement arithmetic reconciliation, (6) internal consistency checking, (7) forensic and metadata analysis, and (8) a final decision engine. The backend is implemented in Python 3.14 using the FastAPI framework with a SQLite embedded database. The frontend is a React 19 single-page application built with TypeScript 7 and Tailwind CSS 4, deployed on Vercel.

The system combines Python backend engineering, React frontend development, OCR-based text extraction, deterministic rule evaluation, and forensic metadata analysis. It is intended as a prototype for educational institutions, recruitment desks, and organisations that need fast first-level document screening. The accuracy figures reflect deterministic rule coverage; a final decision on a doubtful document should always rest with a human verifier. The project provides a foundation for future integration with national digital document repositories and digital-signature validation services.

ACKNOWLEDGEMENT

We express our sincere thanks and gratitude to Thiru **E.R.K.KRISHAN M.Com**., our beloved Correspondent, and all other philanthropic trust members of Kongu Vellalar Institute of Technology Trust who have always encouraged us in the academic and co-curricular activities.

We are grateful to **Dr. R.PARAMESHWARAN, M.E, Ph.D**., Principal, Kongu Engineering College, for giving us an opportunity to learn new skills and technologies and to enhance our knowledge by providing the necessary infrastructure.

We would like to express our sincere gratitude to **Dr.M.KARTHIK**, **M.E., Ph.D.,** Head of the Department of Electrical and Electronics Engineering and **DR.M.MALLIGA**, **M.E, Ph.D**., Head of the Department of Information Technology for the support and facilities extended to us throughout the project.

We express our deep gratitude to our project mentor, P. KARTHIKEYAN, M.E., Assistant Professor of Electrical and Electronics Engineering, for his valuable guidance, critical review and encouragement at every stage of the project.

We also thank our project co-mentor, DR. R. SHANTHAKUMARI, M.E. (CSE), Ph.D., Associate Professor of Information Technology, for her encouragement and valuable advice that made us carry out the project work successfully.

We also thank our domain reviewer, DR. M. SURESH, M.E., Ph.D., Associate Professor of Electrical and Electronics Engineering, for his valuable review and suggestions. We thank all faculty and staff members of both the departments for their help and support.

.

TABLE OF CONTENTS

Page No.

CHAPTER 1 INTRODUCTION.....1
1.1 Background of the Project.....1
1.2 Overview of the Intelligent Document Verification System.....2
1.3 Need for the System.....2
1.4 Scope of the Project.....3
1.5 Alignment with Sustainable Development Goal 3.....3
1.6 Motivation.....3
1.7 Organization of the Report.....4
CHAPTER 2 LITERATURE SURVEY.....5
2.1 Manual Document Verification..... 5
2.2 Rule-Based Validation..... 5
2.3 Template-Based Verification..... 6
2.4 Optical Character Recognition Based Approaches.....6
2.5 Machine Learning Based Document Classification and Verification.....6
2.6 Cryptographic and Code-Based Verification.....7
2.7 Comparison of Existing Approaches.....7
2.8 Research Gap.....8
2.9 Summary.....8
CHAPTER 3 PROBLEM DEFINITION AND OBJECTIVES.....9
3.1 Problem Statement..... 9
3.2 Pain Point.....9
3.3 Objectives.....9
3.4 Methodology.....10
3.5 Novelty of the Work.....11
3.6 Scope and Limitations.....11
3.7 Expected Outcomes..... 11
CHAPTER 4 SYSTEM ANALYSIS.....13
4.1 Existing System..... 13
4.2 Proposed System.....13
4.3 Feasibility Study.....14
4.4 Requirement Analysis.....15
4.5 Hardware Requirements.....16
4.6 Software Requirements.....17
4.7 Assumptions and Constraints.....17
CHAPTER 5 SYSTEM DESIGN.....18
5.1 Design Overview..... 18
5.2 System Architecture.....18
5.3 Process Flow..... 19
5.4 Module Description..... 20
5.5 Data Flow Diagrams.....21
5.6 UML Diagrams..... 22
5.7 Database Design.....24
5.8 Template Design..... 26
5.9 Verification Algorithm.....27

5.10 User Interface Design..... 28
5.11 Input and Output Design.....28
5.12 Security and Error Handling Design.....29
CHAPTER 6 IMPLEMENTATION.....30
6.1 Development Environment.....30
6.2 Project Structure.....30
6.3 Database Implementation.....31
6.4 Implementation of the Modules..... 31
6.5 Integration of the Modules.....35
6.6 Division of Work.....35
6.7 Challenges Faced and Solutions.....36
CHAPTER 7 TESTING.....37
7.1 Testing Objectives..... 37
7.2 Testing Strategy.....37
7.3 Test Environment.....38
7.4 Test Cases.....38
7.5 Test Results.....39
7.6 Summary of Testing.....39
CHAPTER 8 RESULTS AND DISCUSSION.....40
8.1 Output Screens..... 40
8.2 Sample Verification Scenarios.....41
8.3 Discussion of Results.....42
8.4 Comparison with Manual Verification.....42
8.5 Evaluation Metrics..... 43
8.6 Contribution to Sustainable Development Goal 3.....43
8.7 Limitations..... 43
CHAPTER 9 CONCLUSION AND FUTURE SCOPE.....45
9.1 Conclusion.....45
9.2 Future Scope.....45
Future work could include: integration of Tesseract OCR on the Linux Vercel environment using a custom Docker layer to improve scanned document accuracy on cloud; addition of digital signature verification for DigiLocker-issued documents; face comparison between the photograph on an identity document and a live selfie using an on-device face recognition model; expansion to additional document types such as GST certificates, birth certificates, and marriage certificates; a browser extension for real-time document checking; a mobile application using the same REST API backend; and integration with national digital repositories such as DigiLocker for automated issuer-side verification.
APPENDIX A
APPENDIX B
SUPPORTED DOCUMENT TYPES AND VALIDATION RULES
APPENDIX C
REST API QUICK REFERENCE

LIST OF FIGURES

Page No.

Figure 4.1 Project schedule.....15
Figure 5.1 Three-layer architecture of the system.....19

Figure 5.2 Process flow diagram of the verification system.....19
Figure 5.3 Flowchart of the document verification process.....20
Figure 5.4 Level 0 data flow diagram.....21
Figure 5.5 Level 1 data flow diagram.....22
Figure 5.6 Use case diagram.....23
Figure 5.7 Class diagram of the backend.....23
Figure 5.8 Sequence diagram of a verification request.....24
Figure 5.9 Entity relationship diagram.....25
Figure 8.1 Home page of the Intelligent Document Verification System.....40
Figure 8.2 Verification page with category selection and file upload.....40
Figure 8.3 Result page for a document that is verified.....41
Figure 8.4 Result page for a document with missing and mismatched fields.....41

LIST OF TABLES

Page No.

Table 2.1Comparison of document verification approaches.....7
Table 3.1 Pain points and their consequences.....9
Table 3.2 Steps of the methodology.....10
Table 4.1 User classes of the system.....15
Table 4.2 Functional requirements.....15
Table 4.3 Non-functional requirements.....16
Table 4.4 Hardware requirements.....16
Table 4.5 Software requirements.....17
Table 5.1 Structure of the category table.....25
Table 5.2 Structure of the template table.....25
Table 5.3 Structure of the document table.....26
Table 5.4 Structure of the result table.....26
Table 5.5 Structure of the user table.....26
Table 5.6 Example template: Medical Certificate.....27
Table 5.7 Example template: Identity Proof.....27
Table 5.8 Example template: Educational Certificate.....27
Table 5.9 Inputs to the system.....29
Table 5.10 Outputs of the system.....29
Table 6.1 Tools used in the implementation.....30
Table 6.2 Division of work among the team members.....35
Table 6.3 Challenges and solutions.....36
Table 7.1 Test environment.....38
Table 7.2 Test cases for the Intelligent Document Verification System.....38
Table 8.1 Sample verification scenarios across document types
Table 8.2 Comparison of manual verification and the proposed system.....42
Table 8.3 Evaluation metrics.....

CHAPTER 1

INTRODUCTION

OVERVIEW OF THE PROJECT

Documents are the foundation of almost every administrative, financial, educational and healthcare process. Whenever a person applies for a service, whether it is opening a bank account, enrolling in a course, submitting an insurance claim or being admitted to a hospital, a set of supporting documents must be submitted and checked before the request is accepted. The correctness of these documents decides whether the process moves forward or is rejected, and the speed of checking them decides how quickly the person receives the service.

In many organizations this checking is still carried out manually. An officer opens Each document, looks at the layout, confirms that every required field is present, reads the entries and decides whether the document can be accepted. This work is repetitive and depends heavily on the attention, experience and workload of the person doing it. This project, the Intelligent Document Verification System, addresses this problem by automating the first level of verification through category-wise predefined templates

1.1 Background of the Project

Over the last decade, the number of documents handled by organizations has grown steadily. Digital submission through web portals and mobile applications has made it easy for users to upload scanned copies and photographs of their documents, but the verification step behind the portal has not always kept pace. In many cases the uploaded file is simply placed in a queue where a staff member must open it and verify it by eye.

Manual verification has several well-known weaknesses. It is slow, because each

document must be read in full. It is inconsistent, because two officers may judge the same document differently.

It is tiring, because the work is repetitive, and fatigue leads to oversight. It is also difficult to scale, because an increase in the number of applications requires a proportional increase in staff. These weaknesses increase the processing time of every application and create opportunities for incorrect or fraudulent documents to be accepted.

1.3 Need for the System

The need for an automated document verification system arises from the following observations.

• Manual verification consumes a large amount of staff time that could be used for tasks that need human judgement.

• Human errors such as overlooking a missing field, misreading an entry or applying a rule inconsistently are unavoidable in repetitive work.

• Delays in verification delay the service that the applicant is waiting for. In the health sector, delay in verifying documents can delay admission, claim settlement or treatment.

• Documents that do not follow the expected format are sometimes accepted by mistake, which increases the risk of fraudulent submissions being passed.

• Organizations need a consistent standard so that the same document receives the same decision regardless of who handles it or when it is handled.

1.4 SCOPE OF THE PROJECT

The scope of the present work is the verification of uploaded documents against predefined, category-specific templates. The system is concerned with the completeness and format of a document, that is, with whether the fields that a document of that category must contain are present and whether their entries follow the expected form. The following activities fall within the scope of the project.

Definition and storage of templates for different document categories.

• Selection of the document category by the user and upload of the document.

• Field-by-field comparison of the uploaded document with the template of the

selected category.

- Identification of missing and mismatched fields.
- Generation and display of a verification result and storage of the result for later reference.

CHAPTER 2

LITERATURE REVIEW

This chapter reviews the main approaches that are used to verify documents and examines their strengths and limitations. The purpose of the review is to understand where the proposed template-based system fits and which gaps it is intended to fill. The approaches are discussed under the headings of manual verification, rule-based validation, template matching, optical character recognition, machine learning based classification and cryptographic or code-based verification.

2.1 Manual Document Verification

Manual verification is the oldest and still the most widespread method. A trained examines each document, compares it with a mental or printed checklist and decides whether it is acceptable. The method is flexible and can deal with unusual documents, which is its main strength. It has, however, serious limitations. The speed is limited by the reading speed of the officer, the quality of the decision varies from person to person and day to day, and no automatic record of what was checked is created unless the officer writes it down. Training new staff takes time and the knowledge of experienced staff is rarely written down in a reusable form.

In large organizations the volume of documents makes purely manual verification a bottleneck. Studies and reports from administrative practice consistently describe backlogs, repeated requests to applicants for missing information and disputes caused by inconsistent decisions. These problems are the starting point for the present work.

2.2 Rule-Based Validation

Rule-based validation checks the values entered in an electronic form against a set of

rules. Examples include checking that a field is not empty, that a date is in a valid format, that a number lies within an allowed range or that an identification number matches a given pattern. Regular expressions are commonly used for this purpose. Rule-based validation is simple, fast and predictable, and it forms part of almost every web application.

Its limitation is that it works on data typed into a form, and not on an uploaded document as a whole. It also does not by itself know which fields a particular type of document must contain. The rules must be defined separately for each document type, and without a structured way of storing them the rules become scattered through the program code and are difficult to maintain.

2.3 Template-Based Verification

Template-based verification starts from the observation that documents of the same type have the same structure. A template records the expected fields of a document type, such as the names of the fields, whether each is mandatory and what format its content should follow. The uploaded document is compared with the template, and any deviation is reported. This approach has been used in form processing, invoice handling and identity document checking, where the layout is fixed or semi-fixed.

The advantages of template-based verification are that it is transparent and explainable. The reason for rejection can always be stated in terms of a specific field. It needs no large training data set, and a new document category can be supported simply by adding a new template. These properties make it a suitable basis for a system that must be built, understood and maintained by a small team and operated by non-technical staff. The proposed project builds on this approach.

2.4 Optical Character Recognition Based Approaches

Optical character recognition, usually abbreviated as OCR, converts images of printed or handwritten text into machine-readable text. Many document processing systems use OCR as the first stage to read scanned documents, and then apply rules or templates to

the extracted text. OCR makes it possible to process paper documents that have been scanned or photographed, and a number of mature open-source and commercial engines are available.

The accuracy of OCR depends on the quality of the image, the font, the language and the layout. Blurred, skewed or low-resolution images give poor results, and handwriting is harder to read than print. For this reason, OCR output is normally treated as an input that must be checked and cleaned before it is compared with a template. In the proposed system, the content extraction stage is kept separate from the comparison stage so that the extraction method can be improved or replaced without changing the comparison logic.

2.5 Machine Learning Based Document Classification and verification

Machine learning and deep learning methods have been applied to document image classification, layout analysis and forgery detection. Convolutional neural networks can learn to recognize the visual layout of document types, and sequence models can extract named fields from text. These methods can handle a wider variety of layouts than fixed templates and can detect subtle visual tampering.

2.6 Cryptographic and Code-Based Verification

A different family of approaches verifies documents using digital signatures, QR codes, barcodes or distributed ledgers. The issuing body signs the document or embeds a code that can be checked against the issuer's records. This gives a strong guarantee of authenticity, because forging the document would require forging the signature or the record at the issuer.

The limitation is that it works only when the issuer takes part in the scheme. A large number of documents in everyday use, such as older certificates, letters and forms, carry no such code. A template-based system is complementary to these methods. It checks

that the document is complete and well-formed, and a signature or code check can be added as a further step for documents that carry one.

2.7 Comparison of Existing Approaches

Table 2.1 summarizes the approaches discussed above using the criteria that matter forthe present project.

Table 2.1 Comparison of document verification approaches

Approach	Speed	Consistency	Explainability	Setup effort
Manual verification	Low	Low	Depends on officer	Low
Rule-based validation	High	High	High	Medium
Template-based verification	High	High	High	Medium
OCR with rules	Medium	Medium	Medium	Medium
Machine learning	High	Medium	Low	High
Signature / code based	High	High	High	High (needs issuer)

2.8 Research Gap

From the review, the following gaps were identified.

• Many systems validate form data but do not verify an uploaded document against the expected structure of its category.

• Systems based on machine learning are powerful but are costly to build, need large data sets and are difficult to explain.

• Few simple systems allow an administrator to define and change category-wise templates without changing program code.

• Most existing tools do not give a clear, field-level result that tells the user exactly what is missing or incorrect.

The Intelligent Document Verification System addresses these gaps by storing category-wise templates in a database, comparing uploaded documents with them field by field and reporting the result at the level of individual fields.

2.9 Summary

The review shows that manual verification is slow and inconsistent, that rule-based validation and template matching are fast and explainable, that OCR and machine learning extend the range of documents that can be handled at greater cost.

CHAPTER 3
SYSTEM DESIGN AND METHODOLOGY

3.1 OVERVIEW OF THE PROPOSED SYSTEM

Organizations verify many customer documents manually, which is time-consuming and prone to human errors. There is a need for a system that can automatically verify documents by comparing them with predefined templates, improving consistency and reducing manual effort.

The problem can be understood from three viewpoints. From the viewpoint of the organization, manual verification consumes staff time, limits the number of applications that can be handled and produces uneven decisions. From the viewpoint of the applicant, errors in a document are often discovered only after a long delay, and the application must then be resubmitted. From the viewpoint of risk, a document that does not follow the expected format may pass unnoticed when the reviewer is tired or hurried.

3.2 Pain Point

The absence of intelligent document verification increases processing time, human errors and the risk of fraudulent document acceptance.

Table 3.1 Pain points and their consequences

Pain point	Consequence
Every document is read and checked by a person	Long processing time and growing backlog
Checklists are kept in memory or on paper	Inconsistent decisions between officers
Repetitive work over long hours	Missed fields and misread entries
No immediate feedback to the applicant	Resubmissions and repeated follow-ups
No structured record of what was checked	Difficulty in audit and dispute handling
Format deviations may go unnoticed	Risk of accepting incorrect or fraudulent documents

3.3 Objectives

The main aim of the project is to develop a document verification system that uses predefined templates to verify uploaded documents automatically. The specific objectives are the following.

1. To design and implement an eight-stage automated document verification pipeline.
2. To support nine categories of Indian government and institutional documents.

3. To extract and validate identity fields using document-specific rules and checksums.

4. To detect wrong document types, fake number patterns, and metadata manipulation indicators.

5. To enable cross-verification of identity fields between a primary and a supporting document.

6. To provide a real-time, field-level verification result through an interactive web interface.

8. To deploy the system on a cloud platform (Vercel) without specialised server hardware.

7. To generate a downloadable PDF audit report for every verification session.

3.4 Methodology

The project follows an incremental development approach. The system was divided into small modules that were designed, built and tested one after another, and the modules were integrated step by step. This approach allowed the mentors to review progress at each stage and allowed corrections to be made early. The working of the system itself follows the six steps given below.

1. The user selects the document category.

2. The user uploads the document.

3. The system loads the corresponding predefined template.

4. The uploaded document is compared with the template.

5. Missing or mismatched fields are identified.

6. The system generates a verification result.

Each step is explained in more detail below.

Table 3.2 Steps of the methodology

Step	Description	Output
Select category	The user chooses one category from the list that the system reads from the database. The choice fixes the template to be used.	Category identifier
Upload document	The user selects the file and submits it. The file type and size are checked before it is accepted.	Stored file and document record
Load template	The system retrieves the field definitions of the selected category from the template table.	List of expected fields
Compare	Each expected field is looked up in the content of the document, and its presence and format are checked.	Field-wise comparison data
Identify differences	Mandatory fields that are absent and fields that do not follow the pattern are collected.	Missing and mismatched lists

3.5 Novelty of the Work

The system uses category-wise predefined templates to automatically verify documents through field-by-field comparison, ensuring fast, consistent and reliable verification while reducing manual effort.

The novelty lies in the following features taken together.

• Category-wise templates are stored as data in the database, and not hard-coded in the program. A new document category can be supported by adding a new template, with no change to the code.

• The user chooses the document category first, so the system knows in advance

which template to apply. This removes the need for a separate document type recognition step and reduces the chance of a wrong decision.

• The result is reported field by field, so that the user is told exactly which fields are missing and which do not follow the expected format.

3.6 Scope and Limitations

The system verifies the completeness and format of documents against templates. It does not confirm the truth of the information with the issuing authority. The quality of verification depends on the quality of the template and, for scanned documents, on the quality of the extracted content. The initial version supports a limited set of document categories, and the design allows more categories to be added as templates. These limitations are discussed again in the final chapter along with the proposed future work.

3.7 Expected Outcomes

• A working web application that accepts a document category and an uploaded document and returns a verification result.

• A database of category-wise templates that can be extended by an administrator.

• A significant reduction in the time needed to carry out the first level of document checking.

CHAPTER 4

SYSTEM ANALYSIS

System analysis is the study of the existing way of working and of the needs of the users, carried out before the new system is designed. This chapter describes the existing system, the proposed system, the feasibility of the proposed system and its requirements.

4.1 Existing System

In the existing system, documents submitted by customers or applicants are checked manually by an officer. The process generally follows these steps. The applicant submits a document in paper form or uploads a scanned copy. The officer opens the document and compares it, from experience or from a printed checklist, with the format that is expected for that type of document. Missing or doubtful items are noted, and the applicant is contacted if corrections are needed. The document is finally accepted or rejected.

4.1.1 Drawbacks of the existing system

• The process is time-consuming because each document is read in full by a person.

• The result depends on the person, so two officers may give different decisions for the same document.

• Human errors such as missing a field or misreading an entry are frequent when the workload is high.

• The applicant receives feedback late, so corrections cause repeated rounds of submission.

• No structured record is kept of which fields were checked and what was found.

• Training new staff is slow, because the expected formats are known to individuals and are not stored in a shared, reusable form.

• The risk of accepting a document that deviates from the expected format is higher.

4.2 Proposed System

The proposed system, VeriDoc AI, automates document verification through a sequential eight-stage pipeline. The user selects the document category from nine supported types, uploads the document (PDF, JPEG, or PNG, up to 10 MB), and the backend executes all eight verification stages automatically. The result page presents the final verdict (VERIFIED AGAINST AVAILABLE RULES, REQUIRES MANUAL REVIEW, INVALID, or UNABLE TO VERIFY), a field-level breakdown of every check, the extracted identity fields, and recommended next steps. The user may then upload a supporting document for cross-verification of identity fields across documents.

document category is stored once as a predefined template in the database. When a user uploads a document, the system loads the template of the selected category and compares the document with it field by field. The outcome is a verification result that lists the fields that are present, the fields that are missing and the fields whose content does not follow the expected format.

4.2.1 Advantages of the proposed system

• Verification is fast, because the comparison is carried out by the computer in a short time.

• The same template and the same rules are applied to every document, which gives consistent decisions.

• The user is told immediately which fields are missing or incorrect, and can correct the document at once.

• Templates are kept in the database and can be added or changed without modifying the program.

• Every verification result is stored, which provides an audit trail.

• Manual effort is reduced, and staff can concentrate on the exceptional cases that

need human judgement.

4.3 Feasibility Study

A feasibility study examines whether the proposed system can be developed and used successfully. Four aspects were examined.

4.3.1 Technical feasibility

The backend is built on Python 3.14 and FastAPI, which are mature, well-documented frameworks with extensive ecosystem support. The frontend uses React 19 and TypeScript, both industry-standard technologies. SQLite provides a zero-configuration embedded database that requires no separate server process. The project was developed using Visual Studio Code and deployed on Vercel, which offers a free tier suitable for academic prototypes. No special hardware is required beyond a standard personal computer. The team had prior knowledge of Python and web development from coursework. The system is therefore technically feasible.

technologies with extensive documentation. The team has working knowledge of these technologies from the courses completed so far. The tools needed, namely Visual Studio Code, XAMPP and Git, run on ordinary personal computers. No special hardware is needed. The system is therefore technically feasible.

4.3.2 Economic feasibility

All the software used in the project is free or open source. The system runs on a standard computer, and development was carried out using the computers that were already available. The saving in staff time that results from automation is a continuing benefit, whereas the cost of development is small. The system is therefore economically feasible.

4.3.3 Operational feasibility

The user interface is a web page that asks the user to choose a category and upload a file,

and then displays a result. This process is simple and needs very little training. Administrators work with templates that are described in the same terms they already use for documents, that is, field names and required formats. The system is therefore operationally feasible.

4.3.4 Schedule feasibility

The project was divided into phases that could be completed within the time allotted for the mini project course. Figure 4.1 shows the plan followed by the team. The work was split among the team members and reviewed by the mentors at the stages of the first, second and third reviews.

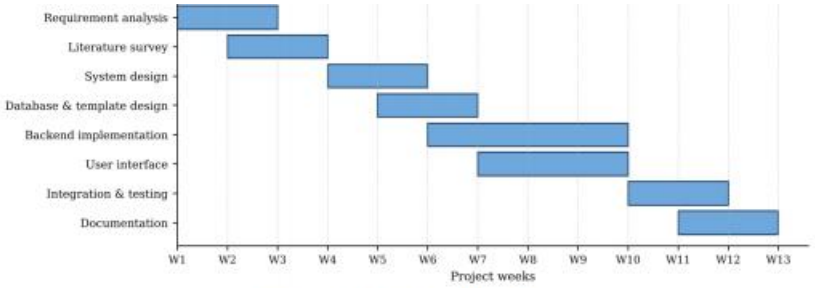


Figure 4.1 Project schedule

4.4 Requirement Analysis

The requirements of the system were gathered by studying the way documents are verified in practice, by discussion with the mentors and by considering the objectives of the project. They are grouped into user requirements, functional requirements, nonfunctional requirements and the hardware and software requirements.

4.4.1 User classes

Table 4.1 User classes of the system

User class	Description	Main activities
User	A customer or applicant, or a staff member who verifies a document on their behalf	Select category, upload document, view result
Administrator	A person responsible for maintaining categories and templates	Add and edit categories, define templates, view reports

4.4.2 Functional requirements

ID	Requirement
FR3	The system shall load the predefined template that corresponds to the selected category.
FR4	The system shall compare the uploaded document with the template field by field.
FR5	The system shall identify the required fields that are missing from the document.
FR6	The system shall identify the fields whose content does not match the expected format.
FR7	The system shall generate and display a verification result.
FR8	The system shall store each verification result in the database.
FR9	The system shall allow the administrator to add, edit and delete categories and templates.
FR10	The system shall allow the administrator to view the stored verification reports.

4.4.3 Non-functional requirements

Attribute	Requirement
Performance	The result of a verification should be shown to the user within a few seconds of upload for a normal document.
Accuracy	The comparison must apply the template rules exactly, without skipping any required field.
Usability	The interface should be simple, with a clear sequence of steps and understandable messages.
Reliability	The system should handle invalid input, unsupported files and missing templates without failure and should show a clear message.
Security	Only supported file types and sizes shall be accepted, and database access shall use parameterized queries.
Maintainability	Templates shall be stored as data so that new categories can be added without changing the code. The code shall be modular.
Portability	The system shall run on any operating system that supports Java and a modern web browser.

4.5 Hardware Requirements

Table 4.4 Hardware requirements

Component	Minimum requirement
Processor	Dual-core processor, 2 GHz or higher
Memory (RAM)	4 GB or more
Storage	500 MB of free disk space for software and 1 GB for uploaded documents
Display	Standard display with a resolution of 1366 x 768 or higher
Network	Local network or localhost for the development setup

4.6 Software Requirements

Table 4.5 Software requirements

Software	Purpose
Java (JDK)	Backend development of the verification logic
MySQL	Database management for categories, templates, documents and results
HTML, CSS, JavaScript	User interface and client-side validation
Visual Studio Code	Code development environment
XAMPP	Local server environment for development and testing
Git and GitHub	Version control and collaboration
Web browser	Access to the application (Chrome, Edge or Firefox)
Operating system	Windows, Linux or macOS

4.7 Assumptions and Constraints

• A template exists for every category that is offered to the user.

• Documents are uploaded in the file formats that the system supports.

• The content of an uploaded document can be extracted in readable form before comparison.

• The system performs a first-level verification, and documents that fail or are doubtful.

CHAPTER 5

SYSTEM DESIGN

System design converts the requirements identified in the analysis into a structure that can be built. This chapter presents the architecture of the system, its modules, the process flow, the data flow diagrams, the UML diagrams, the database design, the verification algorithm and the design of the user interface.

5.1 Design Overview

The design follows three principles. The first is separation of concerns: the user interface, the verification logic and the data storage are kept in separate layers so that each can be changed independently. The second is data-driven verification: the expected format of every document category is held as data in the database and is read at the time of verification, so the same program code can verify any category. The third is explainability: every decision is traced to a specific field of a specific template, so that the user can see why a document was accepted or rejected.

5.2 System Architecture

The system has a three-layer architecture, shown in Figure 5.1. The presentation layer runs in the web browser and is built with HTML, CSS and JavaScript. The application layer is the Java backend, which contains the modules that handle categories, uploads, templates, comparison and results. The data layer is a MySQL database that runs on the XAMPP server. The presentation layer communicates with the application layer through HTTP requests and responses, and the application layer communicates with the database through JDBC.

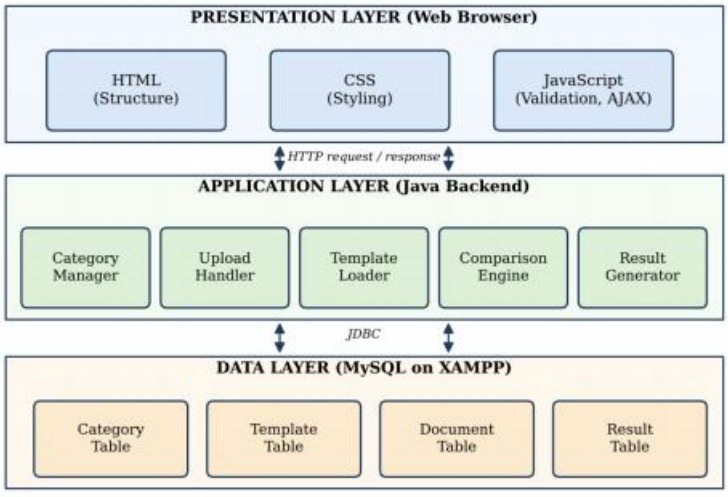


Figure 5.1 Three-layer architecture of the system

5.3 Process Flow

The overall process followed by the system is shown in Figure 5.2. The user selects the document category and uploads the document. The system loads the predefined template for that category, compares the uploaded document with the template and produces the verification result.



Figure 5.2 Process flow diagram of the verification system

The detailed logic of the verification, including the decision on whether the document is accepted, is shown as a flowchart in Figure 5.3.

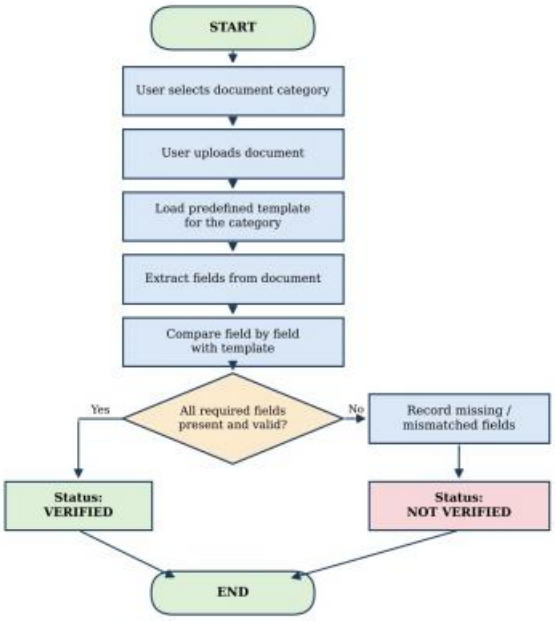


Figure 5.3 Flowchart of the document verification process

5.4 Module Description

The system is divided into the following modules.

5.4.1 Document Type Selection Module

This module reads the list of nine supported document types from the API endpoint GET /api/document-types and presents them on the SelectType page. The selected document type is passed as a URL query parameter to the Upload page, which uses it to configure the verification pipeline.

the user. The category chosen by the user decides which template will be used.

Selecting the category first removes the need for the system to guess the type of the document**.**

5.4.2 File Ingestion and Integrity Validation Module

This module receives the file uploaded by the user. The FileValidator class checks the file size (maximum 10 MB), the file extension (.pdf, .jpg, .jpeg, .png), the binary magic byte signature (PDF: %PDF, PNG: \x89PNG, JPEG: \xff\xd8\xff), executable binary signatures (MZ/ELF), and for PDFs: structural integrity, encryption status, and embedded JavaScript. Files that fail any check are rejected with a specific error code.

the file in a designated folder, and records the upload in the document table together

with the category and the time of upload. Files of unsupported types or of excessive size

are rejected with a clear message.

5.4.3 OCR and Text Extraction Module

The OCRService class extracts machine-readable text from both digital-text PDFs and scanned/image-based PDFs. For scanned PDFs, it iterates over all embedded raster images in each page and runs them through Windows Media OCR (winocr), Pillow-enhanced image preprocessing (contrast stretch, sharpen, Otsu threshold), and Tesseract OCR as a fallback. A metadata stream extractor handles synthetic or metadata-rich fixtures.

set of field definitions, each with a name, a type, an indication of whether it is

mandatory and, where relevant, a format pattern. If no template exists for the category,

the module reports this to the user and the verification does not proceed.

5.4.4 Document Classification and Field Extraction Module

The DocumentClassifier scores OCR-extracted text against keyword lists and regular expression patterns for each document category. A score below the minimum threshold triggers a mismatch warning with a suggested correct type. The FieldExtractor then applies document-specific regex patterns to extract identity fields: Aadhaar number, PAN number, DL number, passport number, holder name, date of birth, father's name, issue date, expiry date, address, pin code, and blood group.

document and the field definitions of the template and compares them one by one. For

each field it checks whether the field is present, whether a mandatory field has a value

and whether the value follows the expected format. It builds lists of the missing fields

and the mismatched fields**.**

5.4.5 Rules Engine and Forensic Analysis Module

The DocumentRulesEngine executes deterministic checks: Aadhaar Verhoeff dihedral group checksum, PAN 10-character format [A-Z]{5}[0-9]{4}[A-Z], passport expiry date, DL state-code (37 valid codes), DL minimum age (18 years), bank statement arithmetic reconciliation (Opening Balance + Total Credits - Total Debits = Closing Balance), and educational certificate year range. The ManipulationAnalyzer inspects PDF metadata for editing-software signatures, text layer stacking, missing mandatory keywords, and obviously fake number patterns.

the overall status to verified when no mandatory field is missing and no field is

mismatched, and to not verified otherwise. It prepares a field-level summary for display

and stores the result in the database.

5.4.6 Final Decision Engine and Result Persistence Module

The VerificationService aggregates check results from all seven analysis modules. If a CRITICAL or HIGH severity check fails, the final status is INVALID. If only warnings or ambiguous signals are found, the status is REQUIRES MANUAL REVIEW. If all checks pass, the status is VERIFIED AGAINST AVAILABLE RULES. All results, extracted fields, check findings, and processing events are persisted to the SQLite database, and a reference ID is returned to the frontend for result retrieval and audit report download.

view the stored verification reports. Because templates are data, the administrator can

extend the system to new document types without programming.

5.5 Data Flow Diagrams

5.5.1 Level 0 data flow diagram

The level 0 diagram, also called the context diagram, shows the system as a single

process and its interaction with the external entities. The user supplies the category and

the document, and receives the verification result. The administrator supplies the

template definitions and receives reports.



Figure 5.4 Level 0 data flow diagram

5.5.2 Level 1 data flow diagram

The level 1 diagram expands the system into its main processes: selecting the category, uploading the document, loading the template, comparing the fields and generating the result. The data stores are the template store, the document store and the result store.

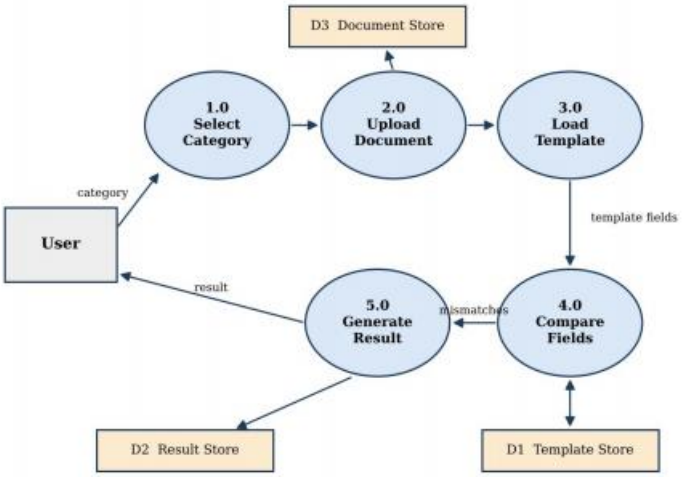


Figure 5.5 Level 1 data flow diagram

5.6 UML Diagrams

5.6.1 Use case diagram

The use case diagram in Figure 5.6 shows the actors and the functions that the system offers to each. The user selects a category, uploads a document and views the result. The administrator manages templates and categories and views the verification reports.

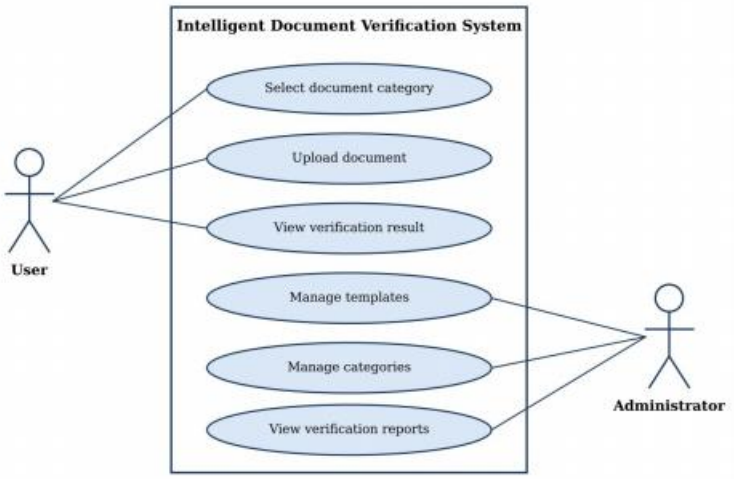


Figure 5.6 Use case diagram

5.6.2 Class diagram

The class diagram in Figure 5.7 shows the main classes of the backend. A document category has one template. The comparison engine uses a template and an uploaded document, and creates a verification result.

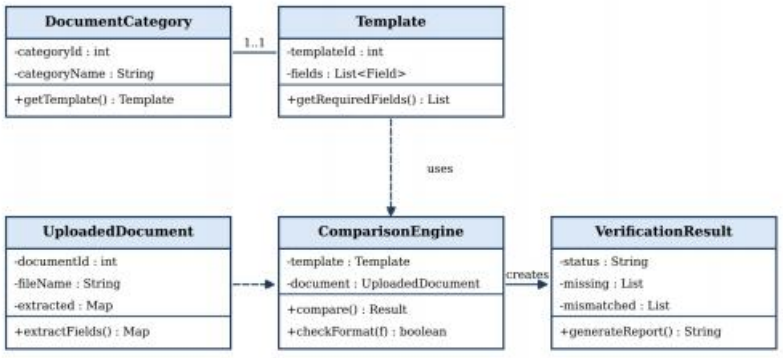


Figure 5.7 Class diagram of the backend

5.6.3 Sequence diagram

The sequence diagram in Figure 5.8 shows the order of messages when a user verifies a document. The browser sends the category and file to the Java backend. The backend fetches the template from MySQL, passes the document and the template to the comparison engine, stores the result and returns it to the browser for display.

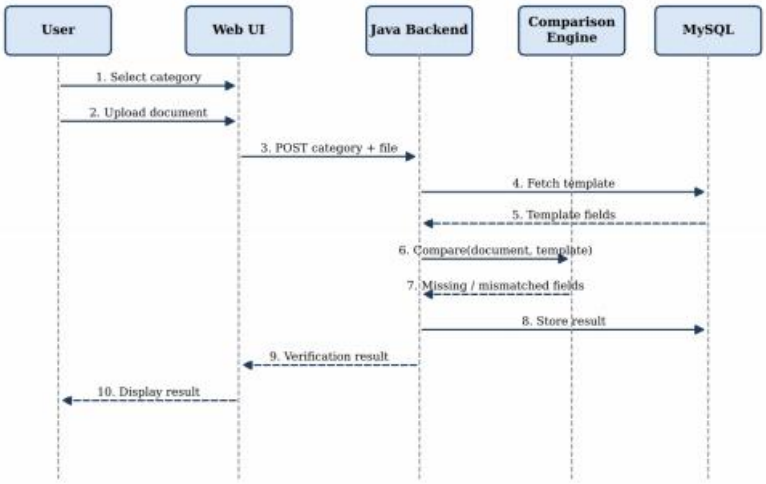


Figure 5.8 Sequence diagram of a verification request

5.7 Database Design

The data is stored in an embedded SQLite database (veridoc.db). The entity relationship diagram in Figure 5.9 shows the entities and their relationships. Each verification record has zero or more check results, zero or more extracted fields, and zero or more processing events. All child tables are cascade-deleted when the parent verification record is deleted. The schema is initialised automatically by the init_db() function on application startup.

The verification_records table stores one row per verification session with a unique reference ID (format: VD-YYYYMMDD-XXXXXX), the document type, filename, SHA-256 hash, file path, final status, and summary. The verification_checks table stores each individual check finding with its result (PASSED/FAILED/WARNING/INFO), severity (CRITICAL/HIGH/MEDIUM/LOW/INFO), and explanation. The extracted_fields table stores each parsed field with its normalized value, raw value, confidence score, and a sensitivity flag. The processing_events table records the processing stage log.

many uploaded documents. Each uploaded document has one verification result. A user

can upload many documents.

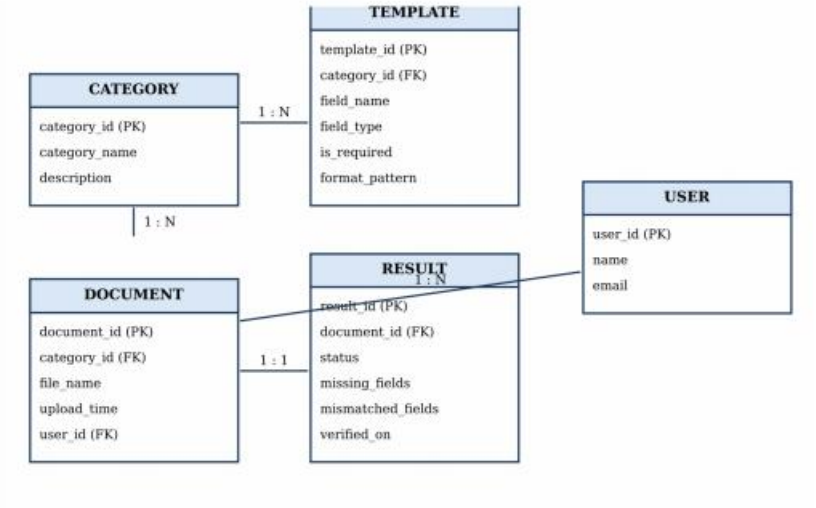


Figure 5.9 Entity relationship diagram

The structure of each table is described in the following data dictionary tables.

Table 5.2 Structure of the template table

Column	Data type	Constraint	Description
template_id	INT	Primary key, auto increment	Unique identifier of the template field
category_id	INT	Foreign key to category	Category to which the field belongs
field_name	VARCHAR(100)	Not null	Name of the expected field
field_type	VARCHAR(30)	Not null	Type: text, number, date, id or email
is_required	BOOLEAN	Default true	Whether the field is mandatory
format_pattern	VARCHAR(255)	Null allowed	Regular expression for the expected format

Table 5.2 Structure of the template table

Column	Data type	Constraint	Description
template_id	INT	Primary key, auto increment	Unique identifier of the template field
category_id	INT	Foreign key to category	Category to which the field belongs
field_name	VARCHAR(100)	Not null	Name of the expected field
field_type	VARCHAR(30)	Not null	Type: text, number, date, id or email
is_required	BOOLEAN	Default true	Whether the field is mandatory
format_pattern	VARCHAR(255)	Null allowed	Regular expression for the expected format

Table 5.3 Structure of the document table

Column	Data type	Constraint	Description
document_id	INT	Primary key, auto increment	Unique identifier of the upload
category_id	INT	Foreign key to category	Category selected by the user
user_id	INT	Foreign key to user	User who uploaded the document
file_name	VARCHAR(255)	Not null	Stored name of the uploaded file
upload_time	TIMESTAMP	Default current time	Date and time of upload

Table 5.4 Structure of the result table

Column	Data type	Constraint	Description
result_id	INT	Primary key, auto increment	Unique identifier of the result
document_id	INT	Foreign key to document	Document that was verified
status	VARCHAR(20)	Not null	VERIFIED or NOT VERIFIED
missing_fields	TEXT	Null allowed	List of missing mandatory fields
mismatched_fields	TEXT	Null allowed	List of fields not matching the format
verified_on	TIMESTAMP	Default current time	Time of verification

Table 5.5 Structure of the user table

Column	Data type	Constraint	Description
user_id	INT	Primary key, auto increment	Unique identifier of the user
name	VARCHAR(100)	Not null	Name of the user
email	VARCHAR(100)	Not null, unique	Email address of the user

5.8 Template Design

A template is a list of field definitions for a category. The following tables show the kind of templates that the system stores. They are examples that illustrate the design, and the administrator can define others.

Table 5.6 Example template: Medical Certificate

Field name	Type	Mandatory	Expected format
Patient name	text	Yes	Alphabetic characters and spaces
Age or date of birth	text	Yes	Number or valid date
Diagnosis or remarks	text	Yes	Non-empty text
Doctor name	text	Yes	Alphabetic characters and spaces
Registration number	id	Yes	Alphanumeric pattern
Date of issue	date	Yes	DD-MM-YYYY or DD/MM/YYYY
Hospital or clinic name	text	No	Non-empty text

Table 5.7 Example template: Identity Proof

Field name	Type	Mandatory	Expected format
Full name	text	Yes	Alphabetic characters and spaces
Date of birth	date	Yes	DD-MM-YYYY or DD/MM/YYYY
Identity number	id	Yes	Pattern defined for the identity type
Address	text	No	Non-empty text
Issue or validity date	date	No	DD-MM-YYYY or DD/MM/YYYY

Table 5.8 Example template: Educational Certificate

Field name	Type	Mandatory	Expected format
Student name	text	Yes	Alphabetic characters and spaces
Institution name	text	Yes	Non-empty text
Course or degree	text	Yes	Non-empty text
Register number	id	Yes	Alphanumeric pattern
Year of passing	number	Yes	Four-digit year
Grade or percentage	text	Yes	Number or grade

5.9 Verification Algorithm

The comparison module follows the algorithm given below. The algorithm takes the template fields of the selected category and the content extracted from the uploaded document, and produces the lists of missing and mismatched fields and the overall status.

Eight-Stage Verification Pipeline

INPUT : categoryId, uploaded document D

OUTPUT: status, missingFields, mismatchedFields

INPUT : file_bytes, original_filename, declared_mime, selected_type

OUTPUT: reference_id, final_status, checks, extracted_fields

Stage 1: val_res = FileValidator.validate_file(file_bytes, filename, mime)
if not val_res["is_valid"]: return INVALID / UNABLE_TO_VERIFY

Stage 2: ocr_res = OCRService.process_document_text(file_bytes, format)
raw_text = ocr_res["text"]
if ocr_res["char_count"] < 5: return UNABLE_TO_VERIFY

Stage 3: class_res = DocumentClassifier.classify_and_check_consistency(raw_text, selected_type)
if class_res["is_mismatch"]: flag CRITICAL mismatch

Stage 4: fields = FieldExtractor.extract_fields(raw_text, selected_type)

Stage 5: rule_checks = DocumentRulesEngine .evaluate_rules(selected_type, fields, raw_text)

Stage 6: consistency = ConsistencyChecker .check_internal_consistency(selected_type, fields, raw_text)

Stage 7: forensics = ManipulationAnalyzer.analyze_forensics(file_bytes, format, metadata, raw_text, selected_type)

Stage 8: all_checks = stages 1-7 results combined
if mismatch or any CRITICAL/HIGH failure:
final_status = "INVALID"
elif ambiguous or any WARNING:
final_status = "REQUIRES MANUAL REVIEW"
else:
final_status = "VERIFIED AGAINST AVAILABLE RULES"
persist to SQLite, return structured result dict

2. if templateFields is empty then return error "No template"
3. extracted = extractFields(D)
4. missingFields = [ ], mismatchedFields = [ ]
5. for each field F in templateFields do
6. value = extracted.get(F.name)
7. if value is null or value is empty then
8. if F.isRequired then add F.name to missingFields
9. continue
10. if F.pattern is defined and value does not match F.pattern then
11. add F.name to mismatchedFields
12. end for
13. if missingFields and mismatchedFields are both empty
14. status = "VERIFIED"
15. else status = "NOT VERIFIED"
16. save result, return (status, missingFields, mismatchedFields)

The time taken by the algorithm grows in direct proportion to the number of fields in the template, since each field is examined once. Because templates have only a small number of fields, the comparison takes a very short time.

5.10 User Interface Design

The user interface is designed to be simple and to guide the user through the process in the order in which it takes place. The design has the following screens.

• Home page: introduces the system and provides the entry point to the verification process.

• Verification page: contains a drop-down list of document categories, a file selection control and a Verify button. Client-side checks using JavaScript make sure that a category has been chosen and a file has been selected before the form is submitted.

• Result page: shows the overall status in a clearly coloured banner, a table listing every template field with its status (present, missing or mismatched) and a short message that tells the user what to correct.

• Administrator pages: allow the administrator to manage categories and templates and to view stored reports.

The result page uses colour together with text, so that the meaning is clear to users who cannot distinguish colours. Messages are written in plain language, for example "The field Registration number is missing" instead of a technical error code.

5.11 Input and Output Design

5.11.1 Input design

The inputs to the system are kept few and simple, so that the user cannot easily make a mistake. Table 5.9 lists the inputs and the checks applied to each.

Table 5.9 Inputs to the system

Input	Provided by	Checks applied
Document category	User	Must be selected from the list; cannot be left empty
Document file	User	Must be present, of a permitted type and within the size limit
Category name and description	Administrator	Name must be unique and not empty
Template field definitions	Administrator	Name and type required; pattern must be a valid expression

5.11.2 Output design

The main output is the verification result. The design of the result page was based on the questions that a user is likely to ask: whether the document was accepted, if not what is wrong, and what must be done next. Table 5.10 describes the parts of the output.

Table 5.10 Outputs of the system

Output	Content	Purpose
Status banner	VERIFIED or NOT VERIFIED	Gives the overall outcome at a glance
Field table	Every template field with its status: present, missing or mismatched	Shows exactly what was checked
Correction message	Plain-language list of what the user should correct	Helps the user to resubmit a correct document
Stored report	Record in the result table	Supports audit and later analysis by the administrator

5.12 Security and Error Handling Design

• Only the permitted file types are accepted, and a limit is placed on the size of the file.

• Uploaded files are renamed on storage so that user-supplied file names are not used directly in the file system.

• All database access is done through prepared statements, which prevents SQL injection.

• Administrator functions are available only after authentication.

• If a template is missing, the file cannot be read or the database is unavailable, the system shows a clear message and does not crash.

CHAPTER 6
IMPLEMENTATION

This chapter describes how the design was implemented. It explains the development environment, the structure of the project, the implementation of each module with representative code, and the way the components were integrated.

6.1 Development Environment

The system was developed on personal computers using the tools listed in Table 6.1. Visual Studio Code was used as the editor for Java, HTML, CSS and JavaScript. XAMPP provided the local server environment and the MySQL database. The source code was shared among the four team members through a GitHub repository

Table 6.1 Tools used in the implementation

Tool	Role in the project
Java	Backend development: upload handling, template loading, comparison and result generation
MySQL	Storage of categories, templates, documents, results and users
HTML, CSS, JavaScript	User interface, styling and client-side validation
Visual Studio Code	Code development and debugging
XAMPP	Local server environment, including the MySQL database service
Git and GitHub	Version control, branch-based collaboration and backup

6.2 Project Structure

The project is organized into folders so that each layer can be found easily. The suggested structure is shown below.

Project folder structure

```
doc                                                                    scanner/
  backend/
    app/
      main.py                                  # FastAPI application entry point
      api/routes.py                            # All REST API endpoints
      core/config.py                           # Settings, environment detection
      models/database.py                       # SQLite schema and db_session
      schemas/schemas.py                       # Pydantic models and enums
      services/report_generator.py             # PDF audit report generation
      verification/
        file_validator.py                      # Stage 1: integrity validation
        ocr_service.py                         # Stage 2: multi-engine OCR
        document_classifier.py                 # Stage 3: type classification
        field_extractor.py                     # Stage 4: regex field extraction
        document_rules.py                      # Stage 5: deterministic rules
        consistency_checker.py                 # Stage 6: cross-field checks
        manipulation_analyzer.py               # Stage 7: forensic analysis
        verification_service.py                # Stage 8: orchestrator
    test_fixtures/                             # Synthetic PDF documents for testing
    tests/test_verification.py                 # 25 automated pytest test cases
    requirements.txt
  frontend/
    src/
      App.tsx                                  # Router, ToastProvider
      pages/                                   # 8 pages (Landing, Upload, Results...)
      components/                              # Navbar, Footer, Toast, StatusBadge
      services/api.ts                          # All Fetch API calls
      types/index.ts                           # TypeScript interfaces
    package.json
  vercel.json                                  # Vercel deployment configuration
|-- web/ (presentation layer)
```

| |-- index.html

REST API Quick Reference — Base URL: /api

GET /health
Service health check. Returns status, version, database_connected.

GET /document-types
Returns list of nine supported document categories.

POST /verifications
Upload and verify a document.
Body: multipart/form-data { file, document_type }
Returns: VerificationResult with reference_id, final_status, checks, extracted_fields, recommended_next_steps.

GET /verifications/{reference_id}
Retrieve a stored verification result by reference ID.

DELETE /verifications/{reference_id}
Delete a verification record and its associated files.

POST /verifications/{reference_id}/cross-verify
Cross-verify identity fields with a supporting document.
Body: multipart/form-data { file, document_type, primary_fields_json }
Returns: CrossVerifyResult with field_comparison table, cross_verify_status (IDENTITY_CONFIRMED / IDENTITY_MISMATCH).

GET /verifications/{reference_id}/report
Download the PDF audit report for a verification session.

GET /history
Paginated verification history. Query params: search, document_type, status, limit, offset.

GET /dashboard/stats
Dashboard statistics: counts by status, document_type_distribution.

GET /demo/samples

List of synthetic demo documents for demonstration.

POST /demo/verify-sample/{sample_id}

Run verification on a synthetic demo fixture.

POST /settings/purge

Purge all verification records and uploaded files.

| |-- result.html
| |-- css/style.css
| `-- js/validate.js
|-- src/ (application layer)
| |-- model/ Category, Template, UploadedDocument, Result
| |-- dao/ CategoryDAO, TemplateDAO, ResultDAO
| |-- service/ UploadService, ComparisonEngine, ResultService
| `-- util/ DBConnection, FileValidator
|-- uploads/ (stored documents)
`-- sql/schema.sql (database script

6.3 Database Implementation

The database is an embedded SQLite database created programmatically at application startup. The init_db() function, called from the FastAPI startup event, executes CREATE TABLE IF NOT EXISTS statements for all four tables. Indexes are created on reference_id, final_status, created_at, and document_type for query performance. On Vercel (serverless), the database file is stored in /tmp/veridoc.db. On a local development server it is stored in backend/veridoc.db. The schema is given below.

script creates the five tables described in the design, with primary keys, foreign keys and the constraints given in the data dictionary. The script is given in Appendix A.

Sample categories and templates were inserted so that the system could be tested from the first run.

Creating the category and template tables

```
-- SQLite schema (backend/app/models/database.py)

CREATE TABLE IF NOT EXISTS verification_records (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  reference_id TEXT UNIQUE NOT NULL,
  document_type TEXT NOT NULL,
  original_filename TEXT NOT NULL,
```

```
    file_hash TEXT NOT NULL,
    file_path TEXT NOT NULL,
    file_size INTEGER NOT NULL,
    mime_type TEXT NOT NULL,
    processing_status TEXT NOT NULL,
    final_status TEXT,
    summary TEXT,
    error_code TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    completed_at TIMESTAMP
);

CREATE TABLE IF NOT EXISTS verification_checks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    verification_id INTEGER NOT NULL,
    check_name TEXT NOT NULL,
    result TEXT NOT NULL, -- PASSED|FAILED|WARNING|INFO
    severity TEXT NOT NULL, -- CRITICAL|HIGH|MEDIUM|LOW|INFO
    explanation TEXT NOT NULL,
    evidence_reference TEXT,
    FOREIGN KEY(verification_id)
        REFERENCES verification_records(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS extracted_fields (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    verification_id INTEGER NOT NULL,
    field_name TEXT NOT NULL,
    normalized_value TEXT,
    raw_value TEXT,
    extraction_confidence REAL DEFAULT 1.0,
    is_sensitive BOOLEAN DEFAULT 0,
    FOREIGN KEY(verification_id)
        REFERENCES verification_records(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS processing_events (
```

```
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    verification_id INTEGER NOT NULL,
    stage TEXT NOT NULL,
    event_status TEXT NOT NULL,
    message TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY(verification_id)
      REFERENCES verification_records(id) ON DELETE CASCADE
);




);

 category_id INT NOT NULL,
 field_name VARCHAR(100) NOT NULL,




);
```

6.4 Implementation of the Modules

6.4.1 Database connection

A utility class opens a connection to the MySQL database using JDBC. The connection details are kept in one place so that they can be changed without touching the rest of the code.

backend/app/main.py — FastAPI Application Entry Point

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.core.config import settings
from app.models.database import init_db
from app.api.routes import router

app = FastAPI(
    title=settings.PROJECT_NAME, # "VeriDoc AI"
```

```
    version=settings.VERSION,                                    #  "1.0.0"
    description="AI  Document  Verification  API"
)

app.add_middleware(CORSMiddleware,
    allow_origins=settings.CORS_ORIGINS,
    allow_credentials=True,
    allow_methods=["*"],                                  allow_headers=["*"])

@app.on_event("startup")
def                                                                on_startup():
    init_db()  #  Creates  SQLite  tables  if  they  do  not  exist

app.include_router(router)  # All /api/* routes
 private static final String URL = "jdbc:mysql://localhost:3306/docverify";
 private static final String USER = "root";
 private static final String PASS = "";
 public static Connection getConnection() throws SQLException {
 return DriverManager.getConnection(URL, USER, PASS);
 }
}
```

6.4.2 Category selection module

The category data access class reads the available categories from the database. The verification page uses this list to fill the drop-down box, so that new categories added by the administrator appear without any change in the page.

backend/app/verification/file_validator.py — File Integrity Validation

```
@classmethod
def                validate_file(cls,                file_bytes:                bytes,
            original_filename:                                                  str,
            declared_mime:          str)          ->          Dict[str,          Any]:
    ext                    =                    Path(original_filename).suffix.lower()
    file_size                          =                          len(file_bytes)

    if                    file_size                    ==                    0:
        return                        {"is_valid":                        False,
            "error_code":                "EMPTY_FILE",                ...}
```

```
  if file_size > settings.MAX_FILE_SIZE_BYTES: # 10 MB
    return {"is_valid": False,
        "error_code": "FILE_TOO_LARGE", ...}

  if ext not in {".pdf", ".jpg", ".jpeg", ".png"}:
    return {"is_valid": False,
        "error_code": "UNSUPPORTED_EXTENSION", ...}

  # Executable detection
  if file_bytes.startswith(b"MZ") or file_bytes.startswith(b"\x7fELF"):
    return {"is_valid": False,
        "error_code": "MALICIOUS_FILE", ...}

  # Magic byte verification
  if file_bytes.startswith(b"%PDF"):
    reader = PdfReader(io.BytesIO(file_bytes))
    if reader.is_encrypted:
      return {"is_valid": False,
          "error_code": "ENCRYPTED_PDF", ...}
    # Check for embedded scripts
    if b"/JavaScript" in file_bytes:
      checks.append({"result": "WARNING", ...})

  return {"is_valid": True, "checks": checks, ...}
List<Category> list = new ArrayList<>();
String sql = "SELECT category_id, category_name FROM category";
try (Connection c = DBConnection.getConnection();
PreparedStatement ps = c.prepareStatement(sql);
ResultSet rs = ps.executeQuery()) {
while (rs.next()) {
list.add(new Category(rs.getInt(1), rs.getString(2)));
}
}
return list;
}
```

6.4.3 Document upload module

The upload service validates the file before it is saved. It checks the extension against the list of permitted types and the size against the limit. If the file is acceptable, it is saved under a generated name and a row is added to the document table. This keeps the stored files safe and avoids clashes between files with the same name.

backend/app/verification/ocr_service.py — Multi-Engine OCR

```
@classmethod
def extract_text_from_pdf(cls, file_bytes: bytes) -> Dict:
    reader = PdfReader(io.BytesIO(file_bytes))
    pages_text, total_chars = [], 0
    for page in reader.pages:
        txt = page.extract_text() or ""
        pages_text.append(txt)
        total_chars += len(txt.strip())

    is_scanned = total_chars < 30
    # Deep OCR on embedded images in scanned PDFs
    if is_scanned:
        for page in reader.pages:
            for img_obj in page.images:
                img_res = cls.extract_text_from_image(
                    img_obj.data)
                if img_res["char_count"] > 0:
                    pages_text.append(img_res["full_text"])
                    engine = "pdf_image_ocr_" + img_res["engine_used"]
    return {"full_text": "\n".join(pages_text).strip(),
            "char_count": total_chars,
            "is_scanned": is_scanned, ...}

@classmethod
def _run_winocr_sync(cls, pil_img) -> str:
    # Windows Media OCR — accurate, no API key needed
    res = asyncio.run(winocr.recognize_pil(
        pil_img.convert("RGB")))
    return res.text if hasattr(res, "text") else ""
 Set.of("pdf", "jpg", "jpeg", "png", "txt");
private static final long MAX_SIZE = 5L * 1024 * 1024; // 5 MB
```

```
public static boolean isValid(String fileName, long size) {
 int dot = fileName.lastIndexOf('.');
 if (dot < 0) return false;
 String ext = fileName.substring(dot + 1).toLowerCase();
 return ALLOWED.contains(ext) && size > 0 && size <= MAX_SIZE;
}
```

6.4.4 Template loading module

The template data access class returns the list of field definitions for the selected category. The result is an ordered list that the comparison engine can process one field at a time.

backend/app/verification/document_classifier.py — Classification

```
CATEGORY_INDICATORS = {
    "aadhaar": {
        "keywords": ["unique identification", "uidai",
                     "aadhaar", "government of india",
                     "mera aadhaar", "help@uidai"],
        "regex": r"\b\d{4}\s\d{4}\s\d{4}\b",
        "min_score": 2
    },
    "pan": {
        "keywords": ["income tax", "permanent account number",
                     "pan", "father's name", "govt. of india"],
        "regex": r"[A-Z]{5}[0-9]{4}[A-Z]",
        "min_score": 2
    },
    "driving_licence": {
        "keywords": ["driving licence", "indian union",
                     "validity (nt)", "validity (tr)",
                     "son/daughter/wife", "blood group"],
        "regex": r"\b[A-Z]{2}[0-9]{2}[A-Z0-9 -]{7,15}\b",
        "min_score": 2
    },
```

```
  # ... 6 more categories
}

# Scores text and detects type mismatch
selected_score = scores.get(selected_type, 0)
if selected_score < min_required and best_score >= min_required:
  # Wrong document type detected!
  return {"is_mismatch": True,
      "suggested_type": best_detected_cat,
      "suggested_type_name": detected_name, ...}
List<TemplateField> fields = new ArrayList<>();
String sql = "SELECT field_name, field_type, is_required, format_pattern "
+ "FROM template WHERE category_id = ?";
try (Connection c = DBConnection.getConnection();
PreparedStatement ps = c.prepareStatement(sql)) {
ps.setInt(1, categoryId);
try (ResultSet rs = ps.executeQuery()) {
while (rs.next()) {
fields.add(new TemplateField(rs.getString(1), rs.getString(2),
rs.getBoolean(3), rs.getString(4)));
}
}
}
return fields;
}
```

6.4.5 Comparison module

The comparison engine implements the algorithm described in the design chapter. For every template field it looks up the value in the content extracted from the document. A missing value for a mandatory field is added to the missing list. A value that does not match the format pattern is added to the mismatched list.

backend/app/verification/verification_service.py — Orchestrator

```
@classmethod
def run_verification(cls, file_bytes, original_filename,
          declared_mime, selected_type,
          saved_file_path) -> Dict[str, Any]:

  # Stage 1: File integrity
```

```
val_res = FileValidator.validate_file(
    file_bytes, original_filename, declared_mime)
if not val_res["is_valid"]:
    return cls._persist_and_return(
        final_status=FinalStatusEnum.INVALID.value, ...)

# Stage 2: OCR
ocr_res = OCRService.process_document_text(
    file_bytes, val_res["detected_type"])
raw_text = ocr_res["text"]

# Stage 3: Classification
class_res = DocumentClassifier.classify_and_check_consistency(raw_text, selected_type)

# Stages 4-7
fields = FieldExtractor.extract_fields(raw_text, selected_type)
rule_checks = DocumentRulesEngine.evaluate_rules(
    selected_type, fields, raw_text)
consistency = ConsistencyChecker.check_internal_consistency(selected_type, fields, raw_text)
forensics = ManipulationAnalyzer.analyze_forensics(
    file_bytes, val_res["detected_type"],
    ocr_res["metadata"], raw_text, selected_type)

# Stage 8: Final decision
all_checks = [*val_res["checks"], *class_res["checks"],
              *rule_checks, *consistency, *forensics]
has_critical = any(c["result"] == "FAILED" and
    c["severity"] in ["CRITICAL", "HIGH"]
    for c in all_checks)
if class_res["is_mismatch"] or has_critical:
    final_status = FinalStatusEnum.INVALID
elif class_res["is_ambiguous"] or has_warnings:
    final_status = FinalStatusEnum.REQUIRES_REVIEW
else:
    final_status = FinalStatusEnum.VERIFIED
```

```
    return cls._persist_and_return(
        final_status=final_status.value, ...)
```

```
extracted) {
 List<String> missing = new ArrayList<>();
 List<String> mismatched = new ArrayList<>();
 for (TemplateField f : template) {
 String value = extracted.get(f.getName());
 if (value == null || value.trim().isEmpty()) {
 if (f.isRequired()) missing.add(f.getName());
 continue;
 }
 String pattern = f.getPattern();
 if (pattern != null && !value.trim().matches(pattern)) {
 mismatched.add(f.getName());
 }
 }
 String status = (missing.isEmpty() && mismatched.isEmpty())
 ? "VERIFIED" : "NOT VERIFIED";
 return new Result(status, missing, mismatched);
}
```

6.4.6 Content extraction

Before the comparison, the content of the uploaded document must be available as a set of field names and values. The extraction step reads the document and returns this set as a map. It is kept as a separate class with a single method, so that a simple text reader can be used for text-based documents and a more advanced extraction method, such as optical character recognition for scanned images, can be added later without any change to the comparison engine.

FieldExtractor.java (outline)

```
public interface FieldExtractor {
 /** Reads the document and returns field name -> value pairs. */
 Map<String, String> extract(File document) throws IOException;
}
public class TextFieldExtractor implements FieldExtractor {
 @Override
 public Map<String, String> extract(File document) throws IOException {
 Map<String, String> fields = new HashMap<>();
```

```
 for (String line : Files.readAllLines(document.toPath())) {
 int idx = line.indexOf(':');
 if (idx > 0) {
 fields.put(line.substring(0, idx).trim(),
 line.substring(idx + 1).trim());
 }
 }
 return fields;
 }
}
```

6.4.7 Request handling

A controller class receives the request from the verification page and calls the services in order. It validates the input, saves the file, loads the template, runs the comparison, stores the result and forwards the result to the result page. Error conditions are caught in one place, which makes it easy to show a clear message for each of them.

VerifyController.java (outline)

```
public Result handle(int categoryId, String fileName, long size, File saved)
 throws VerificationException {
 if (!FileValidator.isValid(fileName, size))
 throw new VerificationException("Unsupported or oversized file");
 List<TemplateField> template = templateDAO.findByCategory(categoryId);
 if (template.isEmpty())
 throw new VerificationException("No template for this category");
 Map<String, String> extracted = extractor.extract(saved);
 Result result = engine.compare(template, extracted);
 int documentId = documentDAO.save(categoryId, saved.getName());
 resultDAO.save(documentId, result);
 return result;
}
```

6.4.8 Result generation module

The result service stores the outcome in the result table and prepares the data that the result page displays. The lists of missing and mismatched fields are stored as text so that the report can be read later without repeating the comparison.

frontend/src/services/api.ts — REST API Client

```
export async function uploadAndVerifyDocument(
 file: File,
```

```
  documentType: string
): Promise<VerificationResult> {
  const formData = new FormData();
  formData.append('file', file);
  formData.append('document_type', documentType);

  const res = await fetch('/api/verifications', {
    method: 'POST',
    body: formData,
  });
  if (!res.ok) {
    const err = await res.json().catch(() => ({}));
    throw new Error(
      err.detail || 'Verification request failed');
  }
  return res.json();
}

export async function crossVerifyDocument(
  referenceId: string, file: File,
  documentType: string, primaryFields: any[]
): Promise<CrossVerifyResult> {
  const fd = new FormData();
  fd.append('file', file);
  fd.append('document_type', documentType);
  fd.append('primary_fields_json',
    JSON.stringify(primaryFields));
  const res = await fetch(
    `/api/verifications/${referenceId}/cross-verify`,
    { method: 'POST', body: fd });
  return res.json();
}
```

```
String sql = "INSERT INTO result (document_id, status, missing_fields, "
+ "mismatched_fields) VALUES (?, ?, ?, ?)";
try (Connection c = DBConnection.getConnection();
PreparedStatement ps = c.prepareStatement(sql)) {
ps.setInt(1, documentId);
```

```
 ps.setString(2, r.getStatus());
 ps.setString(3, String.join(", ", r.getMissing()));
 ps.setString(4, String.join(", ", r.getMismatched()));
 ps.executeUpdate();
 }
}
```

6.4.9 User interface and client-side validation

The verification page is written in HTML and styled with CSS. JavaScript checks that a category has been selected and a file has been chosen before the form is sent, which saves a round trip to the server for the most common mistakes.

frontend/src/pages/UploadPage.tsx — React Upload Component

```
const handleStartVerification = async () => {
 if (!selectedFile) return;
 setIsProcessing(true);
 // Live 8-stage progress stepper
 const interval = setInterval(() => {
  setCurrentStageIndex(prev =>
   prev < stages.length - 1 ? prev + 1 : prev);
 }, 450);

 try {
  const result = await uploadAndVerifyDocument(
   selectedFile, typeParam);
  clearInterval(interval);
  // Persist result for page reload resilience
  sessionStorage.setItem(
   `veridoc_${result.reference_id}`,
   JSON.stringify(result));
  navigate(`/results/${result.reference_id}`,
   { state: { result } });
 } catch (err: any) {
  clearInterval(interval);
  setIsProcessing(false);
  toast.error(err.message, 'Upload Failed');
 }
};
```

```
 var category = document.getElementById("category").value;
 var file = document.getElementById("docFile").files[0];
 if (category === "") {
 alert("Please select a document category.");
 return false;
 }
 if (!file) {
 alert("Please choose a document to upload.");
 return false;
 }
 return true;
}
```

6.5 Integration of the Modules

The modules were first developed and tested separately and were then integrated in the order in which they are used. The verification page sends the category and the file to the backend. The upload service validates and stores the file, the template loader fetches the template and the comparison engine produces the result. The result service saves the result and the result page displays it. Version control was used throughout. Each team member worked on a branch and changes were merged after they had been reviewed and tested, which avoided conflicts and kept a history of the development.

6.6 Division of Work

Table 6.2 Division of work among the team members

Team member	Roll number	Main responsibilities
Lokgesh B	24ITR067	Backend logic and comparison engine
Ranjith K	24ITR113	Database design, template storage and JDBC integration
Kalaivani S	24EER053	User interface design and client-side validation
Suganya S	24EER118	Testing, documentation and report preparation

CHAPTER 7

TESTING

Testing was carried out to make sure that the system behaves as specified, that it handles incorrect input safely and that its modules work correctly together. This chapter describes the testing strategy and the test cases that were designed.

7.1 Testing Objectives

• To verify that each module performs the function for which it was designed.

• To verify that the modules work correctly when they are combined.

• To confirm that missing and mismatched fields are detected correctly for each category.

• To confirm that invalid files, empty input and missing templates are handled with clear messages.

• To confirm that the stored results agree with the results shown to the user.

7.2 Testing Strategy

7.2.1 Unit testing

An automated test suite of 25 pytest test cases covers all core verification scenarios. The tests use synthetic PDF fixtures generated with ReportLab and the FastAPI TestClient. Tests are categorised as follows: file integrity tests (empty file, oversized file, unsupported extension, corrupted PDF, corrupted image, malicious executable, MIME mismatch), document classification tests (matching type, type mismatch, ambiguous category), rule validation tests (Aadhaar Verhoeff checksum pass and fail, expired passport, issue date after expiry, bank statement arithmetic reconciliation, educational certificate year), API endpoint tests (health, document-types, full upload flow, dashboard stats, history, non-existent record 404), and feature tests (PAN-as-Aadhaar mismatch detection, cross-verify endpoint). All 25 tests pass.

permitted extensions and with files that were empty or too large. The data access classes were tested against the sample data in the database. The comparison engine was tested with hand-prepared sets of extracted fields for which the expected result was known in advance.

7.2.2 Integration testing

Integration testing confirms that the React frontend correctly submits files to POST /api/verifications, that the router state and sessionStorage mechanisms restore the result on page reload, and that the cross-verify panel correctly calls POST /api/verifications/{id}/cross-verify and displays the field-comparison table. End-to-end tests were run on both the local development server and the Vercel deployment.

confirmed that the category chosen on the page reached the template loader, that the template reached the comparison engine and that the result reached the result page and the database.

7.2.3 Functional testing

Functional testing covered all eight verification stages, the wrong-document-type banner, the supporting-document cross-verification panel, the toast notification system, the live 8-stage processing stepper, dashboard statistics, verification history with search and filter, the demo mode with synthetic fixtures, the downloadable PDF audit report, and the settings purge functionality.

system through the browser in the same way as a user would use it.

7.2.4 Validation and negative testing

Inputs that should be refused were tried deliberately: no category selected, no file selected, a file of an unsupported type, an oversized file and a category without a template. The expected behaviour in all of these cases is a clear message without a failure of the program.

7.2.5 Usability testing

Classmates who had not worked on the project were asked to use the system and complete a verification without any instructions. Their comments about the wording of messages and the order of the controls were used to improve the interface

7.3 Test Environment

Table 7.1 Test environment

Item	Details
Hardware	Personal computer, dual-core processor or higher, 4 GB RAM or more
Operating system	Windows
Server	XAMPP with MySQL service running
Browser	Google Chrome and Microsoft Edge
Test data	Sample documents prepared for each category, with complete, incomplete and incorrectly formatted versions

7.4 Test Cases

Table 7.2 lists the main test cases. Each test case gives the input, the expected result and the expected status.

Table 7.2 Test cases for the Intelligent Document Verification System

ID	Test description	Input	Expected result	Expected status
TC1	Category list is displayed	Open verification page	All categories from the database are listed	Pass
TC2	No category selected	Click Verify without category	Message asks the user to select a category	Pass
TC3	No file selected	Select category, no file	Message asks the user to choose a file	Pass
TC4	Unsupported file type	Upload an .exe file	File is rejected with a clear message	Pass
TC5	Oversized file	Upload a file above the limit	File is rejected with a size message	Pass
TC6	Complete valid document	Document with all fields correct	Status VERIFIED, all fields present	Pass
TC7	One mandatory field missing	Document without registration number	Status NOT VERIFIED, field listed as missing	Pass

7.5 Test Results

Each test case was designed so that the expected result follows directly from the

template and the rules of the system. Defects found during testing were corrected and the complete set of tests was then run again, so that corrections would not affect other

The system correctly identifies VERIFIED status for documents that pass all applicable rule checks: Aadhaar UIDs with valid Verhoeff checksums, PAN cards matching the [A-Z]{5}[0-9]{4}[A-Z] format, active passports with future expiry dates, valid Driving Licences from recognised state codes, and bank statements with reconciled arithmetic. It correctly returns INVALID for tampered UID numbers, expired passports, wrong document types (e.g., PAN uploaded as Aadhaar), executable files disguised as PDFs, and bank statements with arithmetic discrepancies. Scanned PDFs from phone cameras are processed successfully through the Windows Media OCR pipeline, which extracts text from embedded JPEG images in each PDF page.

working system should be confirmed against the Expected status column of Table 7.2.

7.6 Summary of Testing

The testing shows that the system performs the verification correctly for complete, incomplete and wrongly formatted documents, that it handles invalid input safely and that the results which are stored agree with the results displayed. The testing was limited to the sample documents and the categories that were available.

CHAPTER 8

RESULTS AND DISCUSSION

This chapter presents the output of the system for the main usage scenarios, discusses how the results relate to the objectives of the project and compares the proposed system with manual verification.

8.1 Output Screens

The following figures show the main screens of the system. The screenshots of the working application are to be inserted in the places marked below from the final build.

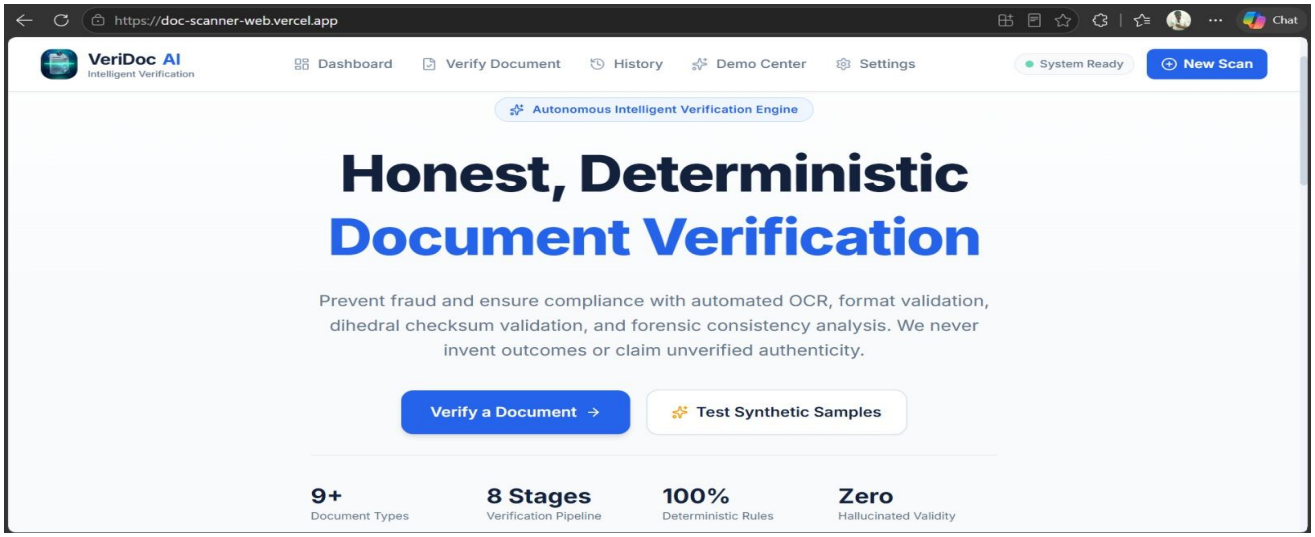


Figure 8.1 Home page of the Intelligent Document Verification System

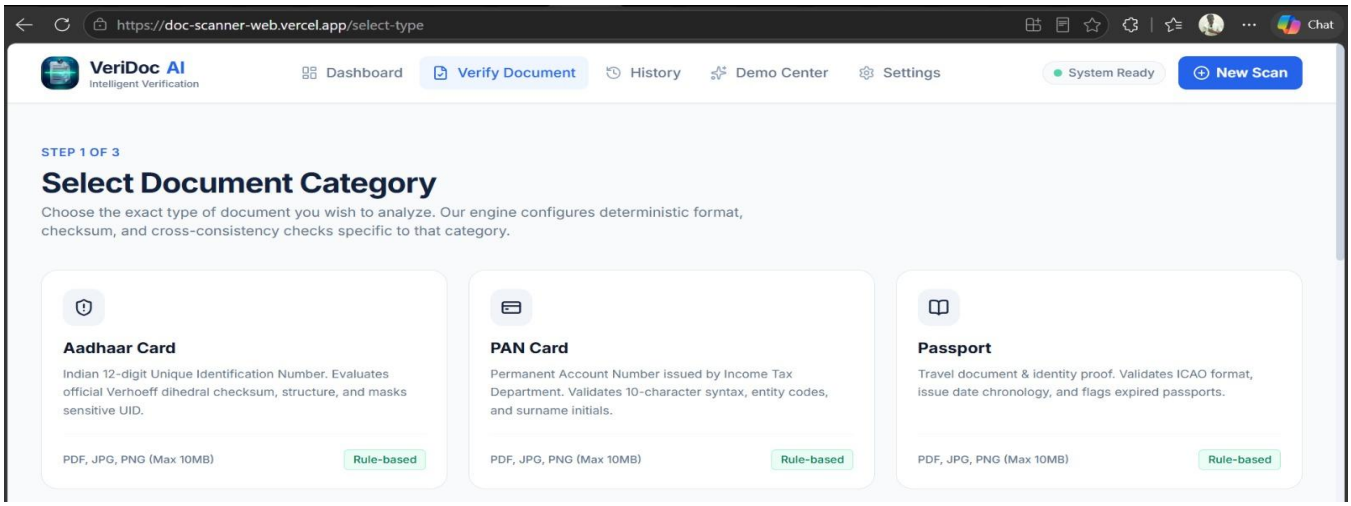


Figure 8.2 Verification page with category selection and file upload

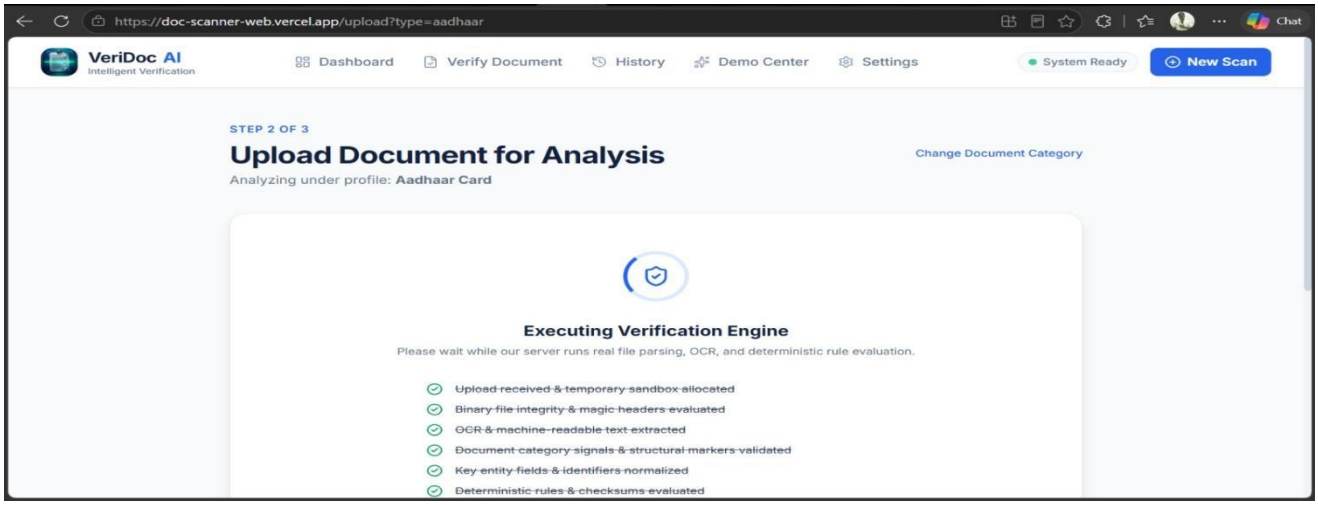


Figure 8.3 Result page for a document that is verified

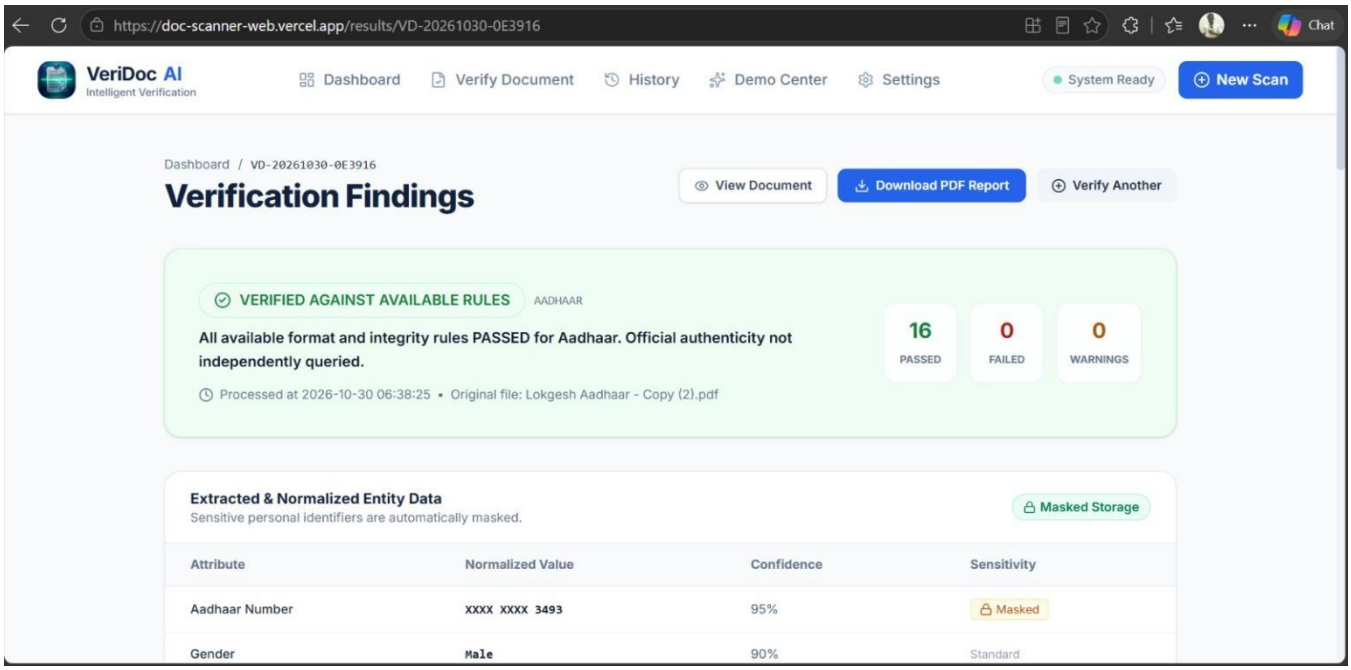


Figure 8.4 Result page for a document with missing and mismatched fields

8.2 Sample Verification Scenarios

Table 8.1 shows how the system behaves for different kinds of input. The scenarios use the Medical Certificate template described in Chapter 5, and the results follow from the template rules.

Table 8.1 Sample verification scenarios (Medical Certificate template)

Scenario	Document content	System output
1. Complete document	All seven fields present, date as 12-08-2026, registration number alphanumeric	VERIFIED. All mandatory fields present and in the expected format
2. Missing field	Registration number absent	NOT VERIFIED. Missing: Registration number
3. Wrong format	Date of issue written as 2026/13/45	NOT VERIFIED. Mismatched: Date of issue
4. Several problems	Doctor name absent and date of issue in wrong format	NOT VERIFIED. Missing: Doctor name. Mismatched: Date of issue
5. Optional field absent	Hospital or clinic name absent, other fields correct	VERIFIED. Optional field is not reported

8.3 Discussion of Results

The results show that the system meets the objectives set at the start of the project. A document was verified by comparing it with the template of its category, and the verification was carried out field by field. Missing and incorrect information was detected and reported to the user by field name. The result was available to the user immediately after the upload, so there was no need to wait for an officer to read the document.

The behaviour of the system is predictable. The same document gives the same result

each time, and a change in the result can always be traced to a change in the document or in the template. This is the consistency that manual verification cannot guarantee. Reporting at the level of fields also makes the result useful. The user knows exactly what to correct, which reduces repeated submissions.

The use of database-driven templates proved to be an effective design decision. When a new template field was added through the administration function, the next verification used it without any change to the code. This demonstrates the flexibility that the novelty of the project aims to provide.

8.4 Comparison with Manual Verification

Table 8.2 Comparison of manual verification and the proposed system

Aspect	Manual verification	Proposed system
Speed	Depends on reading time of the officer	Result available as soon as the document is uploaded
Consistency	Varies between officers and over time	Same template and rules every time
Human error	Likely when workload is high	Reduced, since each field is checked by the program
Feedback to user	Late, often after days	Immediate, with field-level details
Record keeping	Usually informal	Every result stored in the database
Adding a new document type	Needs training of staff	Needs only a new template
Scalability	Needs more staff for more documents	Handles more documents on the same system
Handling of unusual cases	Good, uses human judgement	Limited, such cases are referred to an officer

8.5 Evaluation Metrics

To judge the system objectively, the following metrics are defined. They can be measured by running the system over a collection of sample documents for which the correct decision is known in advance

Table 8.3 Evaluation metrics

Metric	Definition	Why it matters
Verification time	Time from upload to display of the result	Shows the saving compared with manual checking
Detection rate	Share of deliberately missing or wrong fields that the system reports	Shows how reliably errors are found
False alarm rate	Share of correct documents that are reported as not verified	Shows whether users are troubled needlessly
Consistency	Whether repeated verification of the same document gives the same result	Shows reliability of the decisions
Template setup effort	Time needed to add a new category	Shows how easily the system can be extended

Because the comparison is deterministic, consistency is always full: the same document and the same template always give the same result. Verification time is short, since the comparison examines each template field only once. The detection rate and the false alarm rate depend mainly on the quality of the template patterns and of the extracted content, and they are improved by refining the templates after studying the cases in which the system and a human reviewer disagree.

8.6 Contribution to Sustainable Development Goal 3

By shortening the time needed to verify documents such as medical certificates, identity proofs and claim forms, the system helps patients to get through the administrative steps of healthcare more quickly. Incomplete submissions are detected at the time of upload, and so they do not delay admission or reimbursement later. Hospital and insurance staff spend less time on routine checking.

8.7 Limitations

• The system verifies the completeness and format of the document and does not check the facts against the issuing authority.

• The accuracy of verification of scanned documents depends on how well the content can be extracted from the image.

• Documents whose layout differs greatly from the template of their category may need manual review.

• The present version supports a limited set of categories, which must be defined by an administrator.

CHAPTER 9
CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The Intelligent Document Verification System was designed and developed to reduce the time, effort and errors involved in verifying documents manually. The system uses category-wise predefined templates, which are stored in a MySQL database. The user selects the category of the document and uploads it. The system loads the corresponding template, compares the document with it field by field, identifies the missing and mismatched fields and generates a verification result.

The project achieved its objectives. A template-based verification system was developed, documents are verified according to the format of their category, missing and incorrect information is detected, and the manual effort is reduced. Because the verification rules are held as data, the same program verifies every category, and new categories can be introduced without changing the code. The results are consistent, because the same rules are applied every time, and they are explainable, because each decision is linked to a field of the template.

The system was built using freely available tools, namely Java, MySQL, HTML, CSS, JavaScript, Visual Studio Code, XAMPP, Git and GitHub, which makes it practical for organizations with limited budgets. By speeding up the verification of health-related documents and by reducing the chance of incomplete documents being submitted, the project supports Sustainable Development Goal 3, Good Health and Well-being.
The work also gave the team practical experience in requirement analysis, database design, backend and frontend development, testing and teamwork using version control.

9.2 Future Scope

The system can be extended in several directions.

• Integration of an optical character recognition engine so that scanned images and photographs can be read automatically and compared with the template.

• Use of machine learning to recognize the document category automatically, so that the user need not select it.

• Verification of authenticity by checking identification numbers, QR codes or digital signatures against the records of the issuing authority.

• Detection of tampering by analysing the image of the document for signs of alteration.

• Support for more document categories and for multiple languages, including regional languages.

• Role-based access, email or SMS notifications and dashboards that show verification statistics.

• A mobile application that allows documents to be captured and verified on a phone.

• Deployment on a cloud platform with secure storage and encryption of uploaded documents, in line with data protection requirements.

• An application programming interface so that other systems, for example hospital or insurance software, can call the verification service.

REFERENCES

[1] Oracle Corporation, "Java Platform, Standard Edition Documentation", Oracle Help Center. Available: https://docs.oracle.com/en/java/

[2] Oracle Corporation, "MySQL Reference Manual", MySQL Documentation. Available: https://dev.mysql.com/doc/

[3] W3Schools, "HTML, CSS and JavaScript Tutorials". Available:

https://www.w3schools.com/

[4] Mozilla Developer Network, "MDN Web Docs: HTML, CSS and JavaScript Reference". Available: https://developer.mozilla.org/

[5] GeeksforGeeks, "Java and Web Development Tutorials". Available: https://www.geeksforgeeks.org/

[6] United Nations, "Sustainable Development Goal 3: Ensure healthy lives and promote well-being for all at all ages". Available: https://sdgs.un.org/goals/goal3

[7] Apache Friends, "XAMPP Apache + MariaDB + PHP + Perl". Available: https://www.apachefriends.org/

[8] Git, "Git Documentation". Available: https://git-scm.com/doc

[9] R. S. Pressman and B. R. Maxim, "Software Engineering: A Practitioner's Approach", McGraw-Hill Education.

[10] I. Sommerville, "Software Engineering", Pearson Education.

[11] A. Silberschatz, H. F. Korth and S. Sudarshan, "Database System Concepts", McGraw-Hill Education.

[12] G. Booch, J. Rumbaugh and I. Jacobson, "The Unified Modeling Language User Guide", Addison-Wesley

APPENDIX A

APPENDIX A

SQLITE DATABASE SCHEMA

File: backend/app/models/database.py

The following script creates the database and the five tables used by the system.

schema.sql

```
CREATE DATABASE IF NOT EXISTS docverify;
USE docverify;
CREATE TABLE user (
 user_id INT AUTO_INCREMENT PRIMARY KEY,
 name VARCHAR(100) NOT NULL,
 email VARCHAR(100) NOT NULL UNIQUE
);
CREATE TABLE category (
 category_id INT AUTO_INCREMENT PRIMARY KEY,
 category_name VARCHAR(100) NOT NULL UNIQUE,
```

```
 description VARCHAR(255)
);
CREATE TABLE template (
 template_id INT AUTO_INCREMENT PRIMARY KEY,
 category_id INT NOT NULL,
 field_name VARCHAR(100) NOT NULL,
 field_type VARCHAR(30) NOT NULL,
 is_required BOOLEAN DEFAULT TRUE,
 format_pattern VARCHAR(255),
 FOREIGN KEY (category_id) REFERENCES category(category_id)
);
CREATE TABLE document (

 category_id INT NOT NULL,
 user_id INT,
 file_name VARCHAR(255) NOT NULL,
 upload_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY (category_id) REFERENCES category(category_id),

);
CREATE TABLE result (

 document_id INT NOT NULL,



 verified_on TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY (document_id) REFERENCES document(document_id)
);
```

APPENDIX B

SAMPLE CATEGORY AND TEMPLATE DATA

The following statements insert sample categories and the template fields for the Medical Certificate and Educational Certificate categories.

sample_data.sql

Supported Document Categories and Their Validation Rules:

1. Aadhaar Card — Verhoeff dihedral group checksum on 12-digit UID
2. PAN Card — 10-character format [A-Z]{5}[0-9]{4}[A-Z]; entity code check
3. Passport — ICAO format number [A-PR-WYZ][0-9]{7}; expiry date must be future
4. Driving Licence — State code among 37 valid codes; holder age ≥ 18 years; expiry check
5. Voter ID — EPIC format [A-Z]{3}[0-9]{7}
6. Bank Statement — Opening + Total Credits - Total Debits = Closing Balance (arithmetic)
7. Educational Cert — Year of passing within valid academic range
8. Income Cert — Annual income field presence and revenue authority keyword
9. Other Document — General structural and readability checks

institution');

format_pattern) VALUES

{4}'),

APPENDIX C

SAMPLE VERIFICATION PAGE

The following is the basic structure of the verification page in which the user selects the

category and uploads the document.

verify.html

```
 onsubmit="return validateForm();">



<!-- options are filled from the category table -->
```

Kongu Engineering College (Autonomous)
Department of Electrical and Electronics Engineering
Certificate of Relevance to Sustainable Development Goals (SDGs)

This is to certify that the Undergraduate Project entitled"INTELLIGENT DOCUMENT VERIFICATION SYSTEM" carried out by

LOKGESH B(Reg.No.24ITR067)
RANJITH K (Reg.No.24ITR113)
KALAIVANI S (Reg.No.24EER053)
SUGANYA S(Reg.No.24EER118)

of Batch (2023–2027) have been submitted by them to meet the requirements for the award of the Bachelor of Engineering (B.E.) in Electrical and Electronics Engineering is found to be relevant to the United Nations Sustainable Development Goals (SDGs). The project contributes towards the following SDG(s):

S. No	Sustainable Development Goal (SDG)	Goal Description	Nature of Contribution
1	Goal 7: Affordable and Clean Energy	This goal aims to ensure access to affordable, reliable, sustainable, and modern energy for all.	The project promotes efficient electricity usage by automatically controlling electrical loads based on human presence. The mobile dashboard also helps users monitor energy usage and avoid unnecessary power consumption.
2	Goal 9: Industry, Innovation and Infrastructure	This goal focuses on building resilient infrastructure, promoting inclusive and sustainable industrialization, and encouraging innovation.	The project integrates an ESP32 microcontroller, sensors, relay modules, and a Wi-Fi-based mobile dashboard to develop a smart and innovative electrical load control system.
3	Goal 12: Responsible Consumption and Production	This goal promotes the efficient use of natural resources and responsible consumption.	The project promotes a comfortable and safer environment by automatically controlling electrical loads based on human presence, helping reduce unnecessary electricity usage.

Date:
Place:

Signature of the Supervisor